翁恺课 · 第 4 主题

W4 循环 · 练习

对齐翁恺《程序设计入门——C语言》第 4 讲 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

W4-1 填空

填空 **① 一个能正常结束的 `while` 循环通常要凑齐三要素：进入前给变量一个初始值（____）、写一个判断条件（____）、循环体里让条件趋向不成立（____）。请把三要素填到对应位置：`while （____） { ____； ____； }`**

W4-2 填空

填空 **② `for （i = 1； i <= 5； i++）` 中：进入循环前只执行一次的是 ____；每次进循环体前要检查的是 ____；每次执行完循环体后要做的是 ____。把 `i=1`、`i<=5`、`i++` 填进去。**

W4-3 填空

填空 **③ `for （i = 1； i <= 10； i++） { if （i == 3） ____； if （i > 6） ____； printf（"%d "， i）； }` ——想让 i=3 时不打印但继续循环，填 ____；想让 i=7 时整个循环结束，填 ____。**

W4-4 改错

改错 **④ 下面程序想用 do-while 从 3 倒数到 1，但有 1 处语法错误导致编译失败。先自己找，再对照答案（提示：注意 do-while 的语句结尾）。**

```
/* debug_wk4_401.c — 从 3 倒数到 1（找 1 处错误）*/
#include <stdio.h>

int main(void)
{
    int n = 3;
    do {
        printf("%d ", n);
        n--;
    } while (n > 0)
    printf("\n");
```

```
    return 0;
}
```

W4-5 编程

编程 ⑤ 用 `for` 计算 1~100 里所有偶数的和（提示：让 i 从 2 开始、每次加 2）。示例输出应为 `2550`。

W4-6 编程

编程 ⑥ 用 `while` 求一个正整数的各位数字之和。示例：输入 `1234` → 输出 `10`（1+2+3+4）。提示：`n % 10` 取个位，`n /= 10` 去掉个位，直到 n 变成 0。

W4-7 专业场景

专业场景 ⑦ ATM 密码校验：正确密码 `1234`，用户最多试 3 次。用 `do-while` 实现：每次读入密码，错误且没满 3 次就再问；3 次全错输出"卡片已锁定"，猜中输出"欢迎使用"。提示：需要一个计数器 `tries`，条件里同时判断"密码错"和"次数不足"。

W4-8 概念题

概念 ⑧ 结合讲义图 W4-2（while 求和流程）：如果输入 `n = 0`，程序会输出什么？循环体执行了几次？这说明 while 的什么特点？如果同样的需求改用 `do-while` 写，`n = 0` 时输出会有什么不同？

参考答案

① 三要素对应：条件里填 `i <= n` 这类判断；循环体里先有"做事"的语句、再有让条件变化的 `i++` 这类更新。典型骨架：`while (i <= n) { sum += i; i++; }`

> **思路：**初始化在 while 之前（`i = 1`），条件写在括号里，更新写在循环体末尾。**常见错法：**把更新写在条件里（如 `while (i++ <= n)`）——能跑但语义混乱，初学者别炫技；或干脆忘了更新 → 死循环。

② 只执行一次的是 `i = 1`（初始化）；每次进循环体前检查的是 `i <= 5`（条件）；每次循环体结束后做的是 `i++`（更新）。

> **思路：**for 的三件套顺序固定：初始化 → 检查条件 → 循环体 → 更新 → 再检查条件……**常见错法：**以为初始化每次都执行——它只在进入 for 时跑一次；条件为假时循环体一次都不跑。

③ 第一个空填 `continue`（跳过本次，不打印 3 但继续循环）；第二个空填 `break`（i=7 时整个退出）。

> **思路：**continue = 跳过"本次循环体剩余部分"，break = 跳出"整个循环"。**常见错法：**把两个开关记反——continue 也会让"本次"的 printf 不执行，但循环没结束；break 则直接结束循环，后面的 i=8、9、10 全都不再出现。

④ **错误：**

1. 第 10 行 `} while (n > 0)` 结尾**忘了分号**——do-while 语句的结束分号是语法的一部分，不能省。gcc 实测报：

```
wk4_bug_semi.c:10:20: error: expected ';' before 'printf'
   10 |     } while (n > 0)        /* 忘了分号！ */
      |                    ^
      |                    ;
```

2. 修正：在 `while (n > 0)` 后面补上 `;`，程序即可编译运行，输出 `3 2 1`。

> **诊断口径：**do-while 忘分号是**语法错误（error）**，编译直接失败，不是警告。gcc 会指向 `while (n > 0)` 后、下一语句之前的位置，提示 `expected ';'`。看到这个报错，第一反应就是检查 do-while 结尾分号。

⑤ 参考代码：

```
wk4_p5_even.c
#include <stdio.h>

int main(void)
{
    int sum = 0, i;
    for (i = 2; i <= 100; i += 2) {
        sum += i;
    }
    printf("1~100 偶数和 = %d\n", sum);
    return 0;
}
```

实跑输出：

```
1~100 偶数和 = 2550
```

> **思路：**偶数的"步长"是 2，所以初始化 i=2、更新用 `i += 2`，条件 `i <= 100`。**常见错法：**用 `i++` 数 1~100 再在循环里判断 `i % 2 == 0`——能算对，但多跑 50 次；直接用步长 2 更干净。

⑥ 参考代码：

```
wk4_p6_digits.c
#include <stdio.h>

int main(void)
{
    int n, sum = 0;
    printf("输入整数：");
    scanf("%d", &n);

    while (n > 0) {
        sum += n % 10;   /* 取个位 */
        n /= 10;         /* 去掉个位 */
    }
    printf("各位数字之和 = %d\n", sum);
    return 0;
}
```

输入 1234，实跑输出：

```
输入整数：各位数字之和 = 10
```

思路："剥洋葱"：每次取个位（n % 10）加进 sum，再把个位砍掉（n /= 10），直到 n 变 0。这个"取个位→砍个位"的套路是数位拆解的通用手法。**常见错法：**把 n % 10 写进 sum 后忘了 n /= 10 ——死循环；或条件写成 n >= 10 ——个位数时提前退出漏掉最高位。

⑦ 参考代码：

```
wk4_p7_atm.c
#include <stdio.h>

int main(void)
{
    int pwd, tries = 0;
    do {
        printf("输入密码：");
        scanf("%d", &pwd);
        tries++;
    } while (pwd != 1234 && tries < 3);

    if (pwd == 1234) {
        printf("密码正确，欢迎使用\n");
    } else {
        printf("3 次错误，卡片已锁定\n");
    }
    return 0;
}
```

依次输入 1111、2222、1234，实跑输出：

```
输入密码：输入密码：输入密码：密码正确，欢迎使用
```

思路：为什么用 do-while？因为"无论如何都得先让用户输一次密码"。循环条件 = "密码还没对 **且** 次数还没到 3"（&&），两个条件任何一个不满足就停。循环结束后再判断最后那次输入对不对，决定"欢迎"还是"锁定"。**常见错法：**用 || 写循环条件（`pwd != 1234 || tries < 3`）——那意味着"密码错或次数不足就继续"，密码一错就永远停不下来；或忘记 `tries++`，3 次限制形同虚设。

⑧ 参考表述：

> **n=0 时：**while 先判断 `i <= 0`（此时 i=1）→ 不成立 → 循环体一次都不执行，直接输出 `1+2+...+0 = 0`。这就是 while"可能一次都不做"的特点。**若改用 do-while：**它会先执行一次循环体（sum += 1; i++ 变成 i=2）再判断，输出会变成 `1+2+...+0 = 1`——即使输入 0 也硬算了一次。这说明 do-while 保证至少执行一次，同样的输入两种循环结果可能不同——选循环类型要看"这个动作是不是无论如何都要先做一次"。

本练习册为自写原创变式题（对齐翁恺《程序设计入门——C语言》第 4 讲），仅作课堂伴读；改错题与编程题参考代码均用 `gcc -Wall -Wextra` 实测（MinGW-W64 gcc 16.1.0），输出为真实运行结果。原视频/教材版权归原作者所有。